

Cellular Playback Diagnosis

Evidence report based on repository history, exact code paths, and device Logcat | 21 September 2026

Scope: Diagnosis only. No code changes were made while preparing this report. Last-good comparison reference used: **1d1a850**.

1. DIFF against last-known-good (1d1a850)

Command used:

```
git diff 1d1a850 -- app/src/main/kotlin/com/vinmusic/innertube/InnerTube.kt app/src/main/kotlin/com/vinmusic/innertube/ExperimentalResolver.kt
```

Semantic changes (line-ending noise ignored):

```
ExperimentalResolver.kt
- connectTimeout(5, TimeUnit.SECONDS)
- readTimeout(5, TimeUnit.SECONDS)
+ connectTimeout(15, TimeUnit.SECONDS)
+ readTimeout(20, TimeUnit.SECONDS)
```

```
InnerTube.kt
+ WEB_REMIX client (client id 67)
+ non-JSON visitor response guard and HTML fallback
+ network-aware stream URL cache
+ retry-race per-client error logging
+ WEB_REMIX request headers
+ VideoItem.viewCount model field
```

The raw diff was 339.5 KB because of CRLF/LF changes. The semantic diff was 172 lines / 9,417 bytes.

2. Cellular branch

There is no separate cellular /player flow. InnerTube only detects transport for memory-cache validation.

```
private fun currentNetworkKey(): String {
    val context = appContext ?: return "unknown"
    val cm = context.getSystemService(Context.CONNECTIVITY_SERVICE)
        as? ConnectivityManager
    val network = cm?.activeNetwork
    val caps = network?.let { cm.getNetworkCapabilities(it) }
    return when {
        caps == null -> "unknown"
        caps.hasTransport(NetworkCapabilities.TRANSPORT_WIFI) -> "wifi"
        caps.hasTransport(NetworkCapabilities.TRANSPORT_CELLULAR) -> "cellular"
        caps.hasTransport(NetworkCapabilities.TRANSPORT_ETHERNET) -> "ethernet"
        else -> "other"
    }
}
```

The key is checked only before returning a cached signed stream URL. Fresh /player requests use the same code path on Wi-Fi and cellular.

3. ExperimentalResolver timeout

Current file: app/src/main/kotlin/com/vinmusic/innertube/ExperimentalResolver.kt, lines 17-20.

```
.connectTimeout(15, TimeUnit.SECONDS)
.readTimeout(20, TimeUnit.SECONDS)
```

Old value at 1d1a850: 5 seconds connect and 5 seconds read. These are per individual OkHttp /player call; there is no callTimeout or withTimeout. Timeout is caught locally:

```
} catch (e: Exception) {
    Log.e(TAG, "Experimental Resolver Error: ${e.message}")
}
return null
```

The caller receives null and then PlayerSingleton logs Stream URL is NULL.

4. Race loop

Both the initial and refreshed client races have try/catch/finally in each launch block. An ordinary client exception is caught locally, so it does not cancel siblings.

```
CLIENTS.forEach { client ->
    launch(Dispatchers.IO) {
        try {
            val res = fetchViaClient(videoId, client, quality, racingHttp)
            if (!res.isNullOrEmpty()) channel.trySend(res)
        } catch (e: Throwable) {
            log("${client.name} threw in race: ${e.javaClass.simpleName}: ${e.message?.take(80)}")
        } finally {
            if (remaining.decrementAndGet() == 0) channel.close()
        }
    }
}
```

The refreshed race previously swallowed errors with catch (_: Throwable); current working-tree diagnostics log those errors.

5. Per-client visibility

The actual list has five clients, not four: ANDROID_VR, WEB_REMIX, TVHTML5_SIMPLE_EMBEDDED_PLAYER, IOS, and ANDROID_TESTSUITE.

```
log("Trying parallel race: ${client.name}...")
log("playabilityStatus: $status")
log("reason: $reason")
log("${client.name}: LOGIN_REQUIRED ...")
```

Start, exception, status, and LOGIN_REQUIRED logs exist. A successful client result is not explicitly named; it logs only Found direct adaptive audio format. A minimal logging-only suggestion (not applied):

```
val result = runCatching {
    fetchViaClient(videoId, client, quality, racingHttp)
}.getOrNull()
log("${client.name} result=" +
    if (result.isNullOrBlank()) "EMPTY" else "SUCCESS")
```

6. Visitor data

ensureVisitorData() does not track networkKey, transport, IP, or IP changes. It refreshes on force=true, on the one-hour TTL, or when the token is empty.

```
private const val VISITOR_TTL_MS = 60 * 60 * 1000L

private val http = OkHttpClient.Builder()
    .connectTimeout(20, TimeUnit.SECONDS)
    .readTimeout(30, TimeUnit.SECONDS)
```

Both the official visitor_id request and the music.youtube.com HTML scrape use this http client. HTML fallback runs when the official response has no visitorData. If the official request throws, the outer catch logs the exception and does not continue to HTML fallback.

Force refresh happens on LOGIN_REQUIRED, after the first race fails, and at initial getStreamUrl() through ensureVisitorData().

7. History

ExperimentalResolver was introduced in commit [63082bd747a4143dcd8dd6725b317f7c599de9d](#).

```
Commit: Migrate to GitLab CI, secure keystore, and clean up repository
Date: Sat Jun 13 01:02:05 2026 +0530
File: ExperimentalResolver.kt (102 insertions)
```

The commit message does not mention cellular fallback or mobile data. The source comment says it is a Metrolist IOS fallback. Initial clientVersion was 21.03.1 and initial timeout was 5s/5s. WEB_REMIX is an uncommitted working-tree addition.

8. Build

Installed artifact facts: RELEASE APK, versionName 2.3.1, versionCode 21, install Success, relaunch PID 28023.

```
buildTypes {
```

```

    release {
        isMinifyEnabled = true
        proguardFiles(
            getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro"
        )
    }
}

```

shrinkResources is not configured in the shown release block. Gson dependency is implementation(libs.gson). ProGuard rules keep Gson, Gson annotations, and all com.vinmusic.innertube classes:

```

-keep class com.google.gson.** { *; }
-keepattributes Signature
-keepattributes *Annotation*
-keep class com.vinmusic.innertube.** { *; }

```

Manifest includes INTERNET and ACCESS_NETWORK_STATE permissions.

9. Intent and Wi-Fi evidence

The timeout change was intended to give the cellular fallback more time after observed Experimental Resolver timeout. WEB_REMIX was added to recover from cellular ANDROID_VR LOGIN_REQUIRED. networkKey was intended only to prevent reuse of Wi-Fi signed URLs on cellular.

Available device evidence showed Wi-Fi playback working and cellular failure:

```

ANDROID_VR: LOGIN_REQUIRED – forcing visitor token refresh + immediate retry
Experimental Resolver Error: timeout
Stream URL is NULL for Zbnv6sSFksc

```

No original Wi-Fi playback failure was established. The observed issue was specifically cellular playback resolution; Home queue behavior is a separate path.

Conclusion: There is no cellular-only branch in InnerTube and no DoH/DNS change in the inspected diff. The evidence points to YouTube /player client enforcement (LOGIN_REQUIRED) plus the ExperimentalResolver timeout path. Queue and networkKey cache changes are separate from the fresh /player request path.